

Experiment # 08

Creating logical codes of C++ for implementing Stacks using Arrays and Linked list

Objective:-

The objective of this session is to understand the various operations on stacks in C++.

1. Push
2. Pop
3. Traversal

Software Tools:-

1. DevC++11.

Theory:

Theory:-

Stacks are the most important in data structures. The notation of a stack in computer science is the same as the notion of the Stack to which you are accustomed in everyday life. For example, a recursion program on which function call itself, but what happens when a function which is calling itself call another function. Such as a function 'A' call function 'B' as a recursion. So the firstly function 'B' is call in 'A' and then function 'A' is work. So this is a **Stack**. This is a Stack is **First In Last Out** data structure.

Insertions in Stack:

In Stacks, we know the array work, sometimes we need to modified it or add some element in it. For that purpose we use insertion scheme. By the use of this scheme we insert any element in Stacks using array. In Stack we maintain only one node which is called **TOP**. And **Push** terminology is used as insertions.

Deletion in Stack:

In the deletion process, the element of the Stack is deleted on the same node which is called **TOP**. In stacks, it's just deleting the index of the TOP element which is added at last. In Stacks **Pop** terminology is used as deletion.

Display of Stack:

In displaying section, the elements of Stacks are been display by using loops and variables as a reverse order. Such that, last element is display at on first and first element enters display at on last.

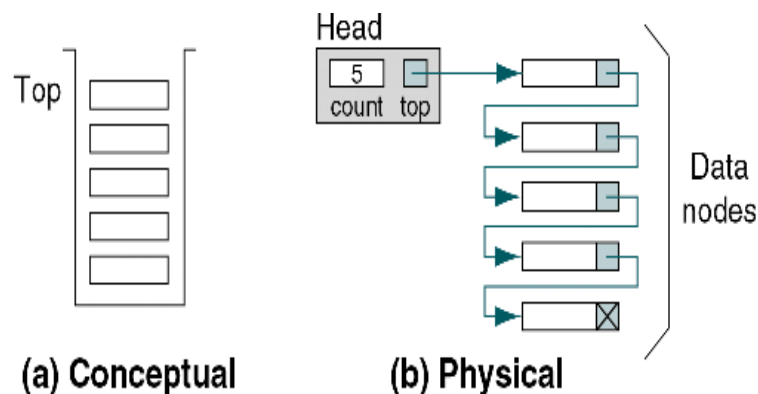
Static Approach Algorithm for Stack Operations

```

Declare and initialize necessary variables, eg top = -1,      MAXSIZE etc.
For push operation
top = MAXSIZE - 1
if (top >= MAXSIZE)
    print "stack overflow"
else
    top = top + 1;
    add item from user
    stack[top] = item
For next push operation, goto step 2.
For pop operation,
if (top < 0)
    print "Stack underflow"
else
    item = stack[top]
    top = top - 1
    Display item
For next pop operation, goto step 4.
Stop

```

Dynamic Approach Algorithm for Stack Operations:



Following figure describes the dynamic approach for stack Implementation.

Step 1)

Declare a **data structure** for the **nodes**.e.g. the following **struct** could be used to create a list where eachnode holds a float –

```

structSNode
{
    int data;
    SNode *next;
}

```

};

Step 2)

Declare a pointer to serve as the stack top, e.g

SNode *top;

Before you use this pointer, make sure it is initialized to NULL.

Once you have done these 2 steps (i.e. declared a node data structure, and created a NULL top pointer, you have an empty stack. The next thing is to implement operations with the stack. Following algorithms describe the push and pop operations on stack.

Push Stack Design (Linked List)

- Create a new node (T)
- Store data in newly created node
- If stack is empty (top=NULL)

make new node as first node of stack and set top=T

- Else

Set the pointer of new node(T) to top

Update top with T.

Pop Stack Design (Linked List)

- If, top=NULL

Display stack is empty and exit.

- Else

Create a temporary stack node (temp)

Set Temp=top;

Set the top to previous node(top=top->next)

Delete temp node.

Tasks:

1. Create stack ADT in C++:-
 - i. Using Static approach
 - ii. Using Dynamic approach